Fidelity Cores: How Much Configuration Does a Security Service Twin Actually Need?

Author Name(s)
Department, Organization
City, Country
email@organization.org

***Abstract*—Security digital twins promise a safe place to test claims about production systems, but a twin is only as good as the configuration it carries. Our earlier platform showed that twins built from defaults give wrong verdicts. This paper asks the narrower question that follows: *which part of a configuration actually decides a security verdict, and what happens when a twin mirrors only some of it?* We introduce three notions. A *fidelity core* is a 1-minimal set of directives that, rendered on top of a base profile, reproduces the source verdict. A *pivot* is a directive whose removal alone flips the verdict. A *guard set* is the set of directive keys whose edits should trigger re-emulation. We measured them on real Redis 7.0, PostgreSQL 16, Nginx 1.24 and OpenSSH 9.6 servers, probed over the wire, across 240 generated configurations, five security properties, 14,400 drift edits and about 42,000 distinct server runs. Every core held at most two directives (median zero; 5% of a configuration on average), yet partial twins were fragile: with 90% of directives mirrored at random, verdicts were still wrong 6.6% of the time. Cores alone were the wrong guide for synchronization, because 39% of pivots lay outside them. A twin built only from learned core-and-pivot keys exported 24% of directives and agreed with the source in 99.3% of held-out cases. A key-guarded resynchronization policy needed five times fewer re-emulations than hash-triggered resynchronization, at 0.28% stale verdict steps. A carefully written static analyzer still disagreed with the real servers in 14 cases. Each disagreement traced to a documented but easily misread rule, in Redis ACL loading or in Nginx's `satisfy any`.**

***Index Terms*—digital twin, configuration fidelity, security validation, delta debugging, configuration drift, service emulation**

I. Introduction

A configuration file is a strange object to copy. Most of it is housekeeping: buffer sizes, log formats, keep-alive timers. A few lines, sometimes one, decide whether a database answers a stranger. A twin that copies everything is faithful but expensive to build, to keep current and to trust with customer data. A twin that copies too little can be confidently wrong. Our previous work on configuration-synchronized service twins [1] made the second danger concrete: twins rendered from vendor-style defaults reached an F1 score of only 0.40 on a declared benchmark, while synchronized twins were correct on every conclusive run. That result settled *whether* configuration matters. It did not say *how much* of it matters, or *which* part.

These questions have practical weight. Each host agent in the platform ships configuration to a shared, multi-tenant control plane, so every exported directive widens the data a breach could expose. Each configuration change triggers re-emulation under the current design, because the agent reports a new component hash, and re-emulation is the most expensive step in the pipeline. And when a twin reports a weakness, an operator wants to know which lines are responsible. All three needs turn on the same unknown: the relationship between individual directives and the security verdict.

This paper studies that relationship directly, on real servers. Our contributions are:

- **Concepts.** We define verdict-relative fidelity for service twins: the fidelity curve, fidelity cores (sufficiency), pivots (necessity) and key guard sets. We also give a simple locality condition under which key-guarded resynchronization can never return a stale verdict (Section III).
- **A measurement harness.** Configurations are rendered onto unmodified Redis, PostgreSQL, Nginx and OpenSSH servers and probed from a non-loopback client with the stock clients. Server output is the only ground truth (Section IV).
- **Measurements** on 240 configurations and five properties, covering partial synchronization, core and pivot structure, selective synchronization, drift and static analysis (Section V).
- **Design lessons** for twin platforms. Synchronize by learned key sets rather than wholesale. Export the absence of a key as data. Guard re-emulation by key, not by hash. Expect even careful static rules to miss rare semantic corners that a real server resolves (Section VI).

The headline numbers pull in two directions, and that tension is the story. Verdicts are *concentrated*: every core we found had at most two directives. They are also *fragile*: dropping a tenth of the configuration at random still flipped one verdict in fifteen. The reconciliation is that the directives keeping a service safe are often not the ones that make it unsafe, and a minimal explanation of a verdict says little about what will break it.

II. Background and Related Work

A. Digital twins for security

The digital-twin idea, a virtual counterpart kept in step with a physical system, grew out of product lifecycle management [2] and spread through manufacturing [3]. Security researchers adopted it for cyber-physical systems [4], [5], and

Dietz and Pernul showed how twins can simulate, replicate and analyze industrial control systems for security purposes [6]. A recent survey maps twin use across security operations [7]. Fidelity is a recurring concern in this literature but is usually treated as a property of the whole twin. We instead measure it per directive and per security property.

Network emulation research has wrestled with the same tension. CrystalNet emulates production networks by running real device firmware with production configurations [8], and its authors note that complete fidelity would require replicating production itself. Cyber ranges make similar trade-offs for training and testing [9]. Our harness follows the "real software, real configuration" philosophy, but asks how little configuration suffices for a given question.

B. Configuration analysis without execution

A different tradition reasons about configuration statically. Batfish derives a network's data plane from router configurations and checks forwarding properties [10]. Config2Spec mines the policies a network configuration implies [11]. Attack-graph tools such as MulVAL encode vulnerability interactions as logic over configuration facts [12], [13]. These approaches are fast and explainable, but they depend on a correct model of each system's semantics. Section V-E measures how far a carefully written model of four services drifts from what the servers actually do.

C. Configuration errors and change

Misconfiguration is a leading cause of failures. Yin *et al.* studied 546 real misconfigurations across commercial and open-source systems [14]. Xu *et al.* found that many configuration knobs are rarely set and add complexity without benefit [15]. ConfErr injects human-like configuration mistakes to test resilience [16]. Sayagh *et al.* surveyed configuration engineering practice and its open problems [17]. Rahman *et al.* catalogued security smells in infrastructure-as-code scripts [18]. At Facebook scale, configuration changes are frequent enough to need their own management system [19]. Our drift model borrows the spirit of these studies: most edits are operational, a minority touch security, and some are rejected by the server outright.

D. Minimization and slicing

Delta debugging reduces a failure-inducing input to a 1-minimal subset that still triggers the failure [20]. Program slicing keeps only the statements relevant to a variable at a point [21]. A fidelity core is a configuration analogue of both. It is the slice of a configuration that preserves one security verdict, and we compute it with delta debugging. We find no prior work that applies verdict-preserving minimization to configurations for twin construction or resynchronization.

III. Concepts

A. Configurations, twins and verdicts

A *configuration* $C = \langle d_1, \ldots, d_n \rangle$ is an ordered sequence of directives. Each directive $d = (\sigma, k, x)$ has a section σ (an Nginx block, an sshd `Match` block, a pg_hba file), a key k and a value x. Order is semantic. pg_hba uses the first matching rule. sshd keeps the first value it obtains for a key. Redis and `postgresql.conf` keep the last. A *base profile* B supplies the values a twin uses for any section-key slot it was not given. We study two profiles. The *vendor* profile V is empty, so unsynchronized slots fall back to compiled-in server defaults. The *platform* profile P reproduces the default-twin settings of our earlier benchmark: Redis protected mode on, SSH password login off, PostgreSQL listening on all addresses with a SCRAM catch-all rule, and Nginx version tokens on.

For a subset $S \subseteq C$ (order preserved), the twin $T_B(S)$ renders S followed by the entries of B whose slot S leaves open. A *property* φ is a wire-level question with a yes/no answer, for example "can an unauthenticated remote client run `CONFIG GET`?" We write $v_\varphi(\cdot)$ for the verdict a real server returns and $v^\star = v_\varphi(T_\mathsf{V}(C))$ for the source verdict. A server that refuses to start yields the verdict *error*, which never equals $v^\star$.

B. Fidelity curve, cores and pivots

Definition 1 (Fidelity curve)**.** *For base B and fraction $p \in [0,1]$, $\varepsilon_B(p)$ is the probability that $v_\varphi(T_B(S)) \neq v^\star$ when S is a uniformly random subset of C of size* $\mathrm{round}(pn)$.

Definition 2 (Fidelity core)**.** *A subset $K \subseteq C$ is a* fidelity core *for (C, B, φ) if $v_\varphi(T_B(K)) = v^\star$ and no $K \setminus \{d\}$ has this property (1-minimality).*

Definition 3 (Pivot)**.** *A directive $d \in C$ is a* pivot *if $v_\varphi(T_\mathsf{V}(C \setminus \{d\})) \neq v^\star$.*

A core answers "what is *sufficient* to reproduce the verdict?" A pivot answers "what is *necessary* to keep it?" The two can differ sharply. Consider a Redis configuration with `protected-mode no` followed by `requirepass`. The source is safe. Under V the empty twin is also safe, so the core is empty. Yet `requirepass` is a pivot, because removing it alone exposes the server.

Remark 1. *Cores are not upward-closed. In the example above, $\emptyset$ is a core but its superset $\{$`protected-mode no`$\}$ gives the wrong verdict. Mirroring "the core plus some more" therefore carries no guarantee. This is why partial twins can be fragile even when cores are tiny.*

Cores are relative to B and need not be unique. We report the one found by ddmin [20] from file order. When even the full twin $T_B(C)$ disagrees with the source, which can happen under P, no core exists, and we record a *base conflict*.

C. Guard sets and locality

A *guard set* G_φ is a set of keys. It drives two policies. *Selective synchronization* ships only $C|_G$, the directives whose key lies in G. *Key-guarded resynchronization* re-emulates after an edit only if the edit adds, removes, modifies or moves a directive with a key in G.

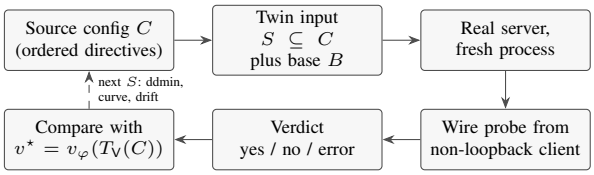


Fig. 1. Measurement loop. Every verdict comes from an unmodified server started from rendered files and probed over TCP with the stock client. Experiments differ only in how S and B are chosen.

Definition 4 (Key locality)**.** *φ is G-*local *over a family of configurations if $C|_G = C'|_G$ implies $v_\varphi(C) = v_\varphi(C')$.*

Proposition 1. *If φ is G-local, then (i) the selectively synchronized twin $T_\mathsf{V}(C|_G)$ has the source verdict, and (ii) key-guarded resynchronization never reports a stale verdict.*

Proof. (i) $T_\mathsf{V}(C|_G)$ renders exactly $C|_G$, and $(C|_G)|_G = C|_G$, so locality gives equal verdicts. (ii) An edit that touches no key in G leaves $C|_G$ unchanged, including the relative order of its directives, because moving a non-G directive does not reorder G directives. Locality then says the verdict is unchanged, so skipping re-emulation is safe. Any edit that touches G triggers re-emulation. □

Two consequences shape the experiments. First, locality is stated against V. Under P, a key that is absent from the source still receives the platform's value, so the twin quietly disagrees whenever the source relied on a vendor default. Second, locality is an empirical property of a configuration family, so a guard set learned from one sample can miss a key that only matters in configurations the sample never contained. Selective synchronization accuracy is a direct test of locality, and we report it as such.

IV. Measurement Harness

A. Servers, scaffolding and probes

Fig. 1 summarizes the loop. Each evaluation writes the rendered files, starts a fresh server, waits for readiness, probes it and stops it. Ports, PID files, data directories, host keys and the static document tree are fixed scaffolding per worker slot, so they never become fidelity variables. The server binds to, and the client connects from, a non-loopback address added to the loopback interface. This matters because Redis protected mode treats loopback clients differently. The five properties are:

- **Redis**: `redis-cli CONFIG GET maxmemory` succeeds without credentials (an administrative command, so ACL categories and renamed commands matter).
- **PostgreSQL**: `psql -w` as role `synthetic` runs `SELECT 1` with no password.
- **Nginx (two properties)**: the `Server` header of `GET /` reveals a version; `GET /admin/` returns 200.
- **OpenSSH**: user `synthetic` logs in with a synthetic password via `sshpass`.

Probes classify only recognized outcomes, such as `NOAUTH`, "no pg_hba.conf entry" or "Permission denied". Anything else is retried once and otherwise marked inconclusive. No inconclusive outcome survived the retry in any experiment. Startup failures were recorded with the server's own message. Versions were Redis 7.0.15, PostgreSQL 16.15, Nginx 1.24.0 and OpenSSH 9.6p1 on Ubuntu 24.04, on a 4-vCPU, 16 GB cloud container. Four worker slots ran in parallel. A verdict cache keyed by the canonical twin avoided repeating identical runs. All credentials and users were synthetic lab values.

TABLE I
Corpus: 60 configurations per family, five properties

Property φ	Keys (sec./ops)	Dir./config	Exposed	Run (s)
Redis unauth. admin	5 / 14	9.0 (5–13)	20	0.13
PG passwordless SQL	3 / 12	10.8 (6–15)	14	0.08
Nginx version leak	6 / 12	10.4 (6–15)	30	0.03
Nginx /admin/ open			31	
SSH password login	10 / 12	10.2 (6–16)	17	0.16

Keys: catalog keys labeled security/operational. Dir./config: mean (range). Exposed: sources with verdict *yes*. Run: median wall time per twin evaluation (start, probe, stop).

B. Configuration corpus

No public corpus pairs service configurations with wire-verified security verdicts, so we generated one from a seeded catalog. For each family, the catalog lists legal sections, plausible values and a class label: *security* or *operational* (Table I). Generation combines 3–10 operational directives with a *posture*, a short recipe modeled on familiar patterns. Examples include protected mode switched off for a container network, a pg_hba trust rule for a CI subnet, a server-wide `deny all` reopened in one location, and an appended sshd override that never takes effect because sshd keeps the first value. Security directives are interleaved at random positions. A candidate is accepted as a source only if the real server starts and answers conclusively. Three of 360 candidates failed, all sshd configurations that demanded a disabled method in `AuthenticationMethods`. The first 60 accepted configurations per family form the corpus, split by index parity into training and held-out halves. The class label is used only for the expert guard set and the drift mix, never when computing cores or pivots.

C. Experiments

E1 (fidelity curve). For each configuration, base and $p \in \{0, 0.1, \ldots, 1\}$, eight random subsets (one at $p \in \{0, 1\}$), giving 44,372 property verdicts. **E2 (cores and pivots).** ddmin cores for every configuration, property and base, plus all single-directive removals under V. **E3 (selective synchronization and drift).** Guard sets learned from the *training* half: G_{core} (keys in any core), G_{pivot} (keys of any pivot) and their union, compared with G_{expert} (all keys labeled security). Each held-out configuration receives three 40-edit traces with a security-edit share $q \in \{0.1, 0.3, 0.5\}$. An edit modifies a directive with probability 1/2, or adds, deletes or moves one with probability 1/6 each; illegal additions are redrawn. An edit that the real server refuses is rolled back, as an operator

would roll it back; this happened 32 times. Every version's source verdict is measured, and resynchronization policies are replayed against those verdicts. **E4 (static analysis).** Two analyzers are evaluated on all 13,606 distinct configurations seen in E0 and E3. The *naive* one mirrors the platform's original static baseline: it looks up the last value of a few keys. The *careful* one encodes documented semantics: last-wins and first-wins keys, pg_hba first match, Nginx per-level inheritance of access rules, sshd `Match` precedence and Redis ACL tokens. Both were written from documentation before any comparison and were not tuned afterwards.

V. Results

A. Partial fidelity is fragile (E1)

Fig. 2 shows that error falls roughly linearly with the share of directives mirrored. There is no early plateau where "most of the configuration" is good enough. Under V, overall error was 45.3% with nothing mirrored, 28.3% at half (95% CI 25.2–31.5%), 11.6% at 80% and still 6.6% (5.5–7.8%) at 90%. Families differ in shape. SSH starts highest (71.7%), because the vendor default allows password login and most sources disallow it in some way. PostgreSQL stays flat near 23% until most rules are present, because a trust rule only matters if `listen_addresses` and every earlier matching rule come along with it.

The direction of error depends on the base. Across partial levels under V, false alarms (twin says exposed, source is safe) outnumbered false-safe verdicts by 3,658 to 1,980. A random cut tends to drop a defensive line and leave an exposure standing. The platform profile P halves the SSH error at low fidelity but never reaches zero. At full synchronization it still erred on 1.3% of verdicts, all for SSH: sources that relied on the vendor default for `PasswordAuthentication` were overridden by the platform's "no". In 88 runs the platform value also made sshd refuse to start, because it contradicted an `AuthenticationMethods` line in the source. A base profile is therefore not a neutral filler. It is configuration the source never had.

B. Cores are tiny; pivots are not inside them (E2)

Across all 300 configuration–property pairs under V, the largest core held two directives, the median was zero and the mean was 0.50, about 5.1% of a configuration (Fig. 3). A core of size two appeared only for PostgreSQL exposures, which need both `listen_addresses` and the first matching trust rule. Under P the PostgreSQL cores shrink to one directive, because the platform already listens on all addresses, while four SSH configurations had no core because of the base conflict described above. Computing a core was cheap: 3.2–5.1 distinct server runs on average and at most 22, typically well under a second of server time.

Small cores do not mean a small relevant set. Table II contrasts cores with pivots. Of 194 pivots, 76 (39%) lay outside the vendor core. For the Nginx `/admin/` property only 22 of 62 pivots were in the core. A configuration that closes the admin path with `deny all` has an empty core whenever its verdict matches the vendor default (open). Yet that `deny all`, the `allow` line that reopens a location and the `satisfy` directive can each flip the verdict when removed. Conversely, 79–91% of directives labeled *security* were in no vendor core. Examples include `PermitRootLogin` and `PubkeyAuthentication` for the password-login property, and rules shadowed by an earlier pg_hba match. A security label is a statement about a key. A core is a statement about one verdict.

TABLE II
Cores (sufficiency) versus pivots (necessity), base V

φ	$\overline{\lvert K\rvert}$	$\lvert K\rvert$=0	Pivots	in K	Inert sec.	Core keys
Redis	0.33	40	43	20	84%	protected-mode
PostgreSQL	0.47	46	33	26	91%	listen, hba
Nginx leak	0.50	30	27	27	84%	server_tokens
Nginx admin	0.48	31	62	22	80%	deny, internal
SSH	0.72	17	29	23	79%	PasswordAuth.

$\lvert K\rvert$=0: configurations (of 60) whose core is empty. Pivots: directives whose single removal flips the verdict; "in K": of those, how many are in the core. Inert sec.: share of security-labeled directives that are in no core. Core keys: most frequent core keys.

C. Selective synchronization (E3)

Table III evaluates twins that receive only the directives whose key is in a guard set learned on the training half. They were tested on held-out configurations and on drift versions of them, 900 verdicts per row. Learned from cores alone, G_{core} for Redis is just `protected-mode`, and accuracy drops to 91.4%: the core never "sees" the directives that keep a service safe. Adding pivot keys raises accuracy to 99.3% while exporting 24% of directives. The six remaining errors involve keys that were never decisive in the training half, Redis `bind` and sshd `DenyUsers`. They are exactly the locality failures Section III predicts for a learned guard. The expert list was locally complete on this corpus (100%) at a higher export cost of 35% of directives.

Under the platform profile, every guard set lost about five points. The expert set's 46 errors were 26 SSH false-safe verdicts (absent `PasswordAuthentication`, filled with "no"), 16 PostgreSQL false alarms (absent `listen_addresses`, filled with "*") and 4 SSH startup errors. A selectively synchronized twin must therefore carry *absence* as data. For every guard key, the agent should report "not set" so that the twin falls back to the vendor default rather than the platform's. With that marker, the twin for these properties reduces to the V rows of Table III.

D. Drift and resynchronization (E3)

The current platform design re-emulates whenever a component hash changes, so in effect after every edit. That is never stale, but nearly all of the work is wasted. With $q = 0.3$, the true verdict flipped on only 4.8 of every 100 edits (Table IV, Fig. 4). Periodic re-emulation is the obvious cheap alternative, and it performed poorly. Every fourth edit

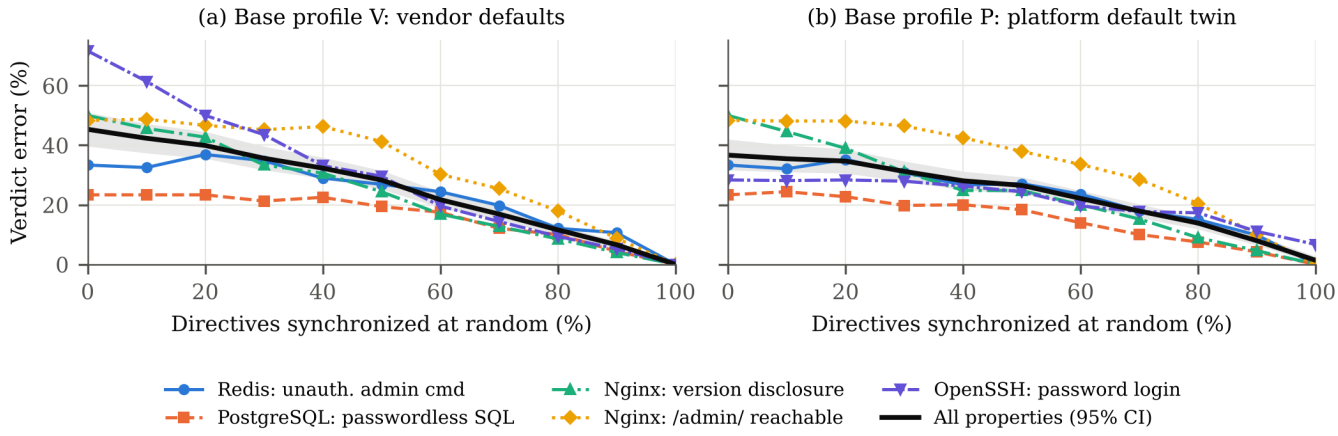


Fig. 2. Fidelity curves. Verdict error of twins that mirror a random fraction of the source directives, per property, on top of (a) vendor defaults and (b) the platform default-twin profile. Black: all properties, with a 95% cluster-bootstrap interval over configurations.

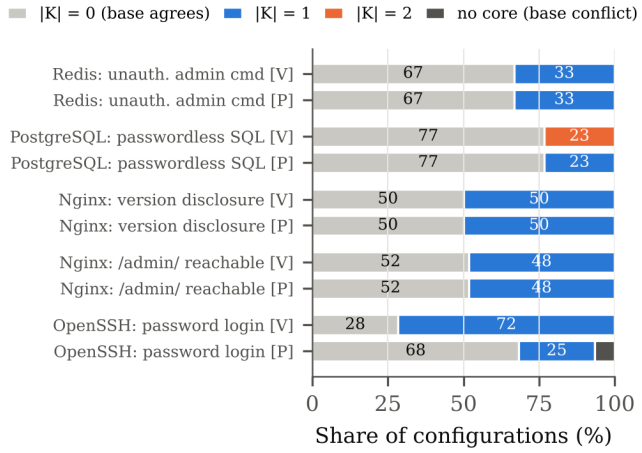


Fig. 3. Fidelity-core sizes under the vendor [V] and platform [P] base profiles. Gray: the base alone already reproduces the source verdict.

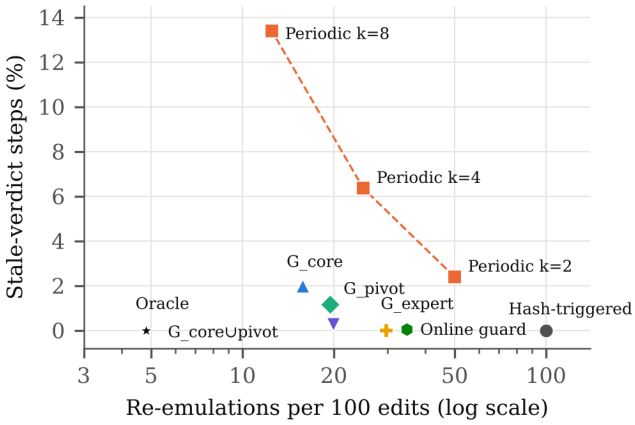


Fig. 4. Cost and staleness of resynchronization policies over 120 held-out drift traces of 40 edits (security-edit share $q = 0.3$). Lower-left is better; the oracle re-emulates only when the true verdict flips.

TABLE III
SELECTIVE SYNCHRONIZATION WITH LEARNED AND EXPERT GUARD SETS

Guard set	Exported	Acc. (V)	95% CI	Acc. (P)
G_{core}	20.5%	91.4%	88.2–94.3	86.4%
G_{pivot}	23.8%	98.4%	97.3–99.4	93.7%
$G_{\text{core}} \cup G_{\text{pivot}}$	24.1%	99.3%	98.7–99.9	94.3%
G_{expert}	34.8%	100%	100–100	94.9%

Exported: mean share of source directives sent to the twin. Accuracy over 900 held-out verdicts (150 configuration–property pairs × 6 versions); CI by cluster bootstrap over pairs.

cost 25 re-emulations per 100 edits and left 6.35% of steps stale, catching 93 of 289 flips immediately.

Key guards did much better. The learned union guard used 19.9 re-emulations per 100 edits, five times fewer than hash-triggered, and left 0.28% of steps stale. It caught 285 of 289 flips at once; the four it missed were `bind` and `DenyUsers` edits. The expert guard was never stale at 29.7 per 100. Guards also adapt to the property. For Nginx version disclosure the learned guard is only `server_tokens`, and it re-emulated on 6.2 of 100 edits, against 30.2 for the expert list. Adding a periodic audit to the learned guard, or growing the guard online whenever an audit finds a stale verdict, cut residual staleness to 0.13% and 0.03%. On this corpus, though, the audits cost about as much as simply using the expert list. The pattern held across edit mixes. At $q = 0.1$ and $q = 0.5$ the union guard needed 6.5 and 32.2 re-emulations per 100 edits, with 1.05% and 0.58% stale steps; the expert guard needed 9.9 and 49.6 with none.

E. Static analysis versus the real servers (E4)

Table V answers the obvious objection: if only one or two directives matter, why not just read them? The naive analyzer, in the style of our platform's original static baseline, was right only 72–93% of the time. It ignored sshd's first-value rule, pg_hba ordering and scoping, and Nginx's rule that any

TABLE IV
RESYNCHRONIZATION POLICIES ($q = 0.3$, 4,800 EDITS, 289 TRUE FLIPS)

Policy	Re-emul./100 edits	Stale steps	Late flips
Hash-triggered (current)	99.8	0.00%	0
Periodic, $k = 2$	50.0	2.38%	143
Periodic, $k = 4$	25.0	6.35%	196
Periodic, $k = 8$	12.5	13.38%	221
Guard G_{core}	15.8	2.00%	24
Guard G_{pivot}	19.5	1.15%	14
Guard $G_{\text{core}} \cup G_{\text{pivot}}$	19.9	0.28%	4
Guard G_{expert}	29.7	0.00%	0
Union guard + audit every 8	29.8	0.13%	4
Online-grown guard	34.8	0.03%	1
Oracle (lower bound)	4.8	0.00%	0

Stale steps: share of versions at which the twin's verdict differs from the source (95% CI for the union guard: 0.00–0.72%). Late flips: true flips not caught at the edit that caused them.

TABLE V
STATIC ANALYZERS AGAINST REAL-SERVER VERDICTS (13,606 CONFIGURATIONS)

	Naive			Careful		
φ	Acc.	FP	FN	Acc.	FP	FN
Redis unauth. admin	88.4%	392	4	99.82%	2	4
PG passwordless SQL	74.5%	868	0	100%	0	0
Nginx version leak	92.7%	157	94	100%	0	0
Nginx /admin/ open	76.1%	127	691	99.77%	8	0
SSH password login	72.3%	843	90	100%	0	0

access directive at an inner level replaces the parent's whole list. The careful analyzer, which encodes those rules, agreed with the servers on 17,018 of 17,032 verdicts. Its 14 errors are instructive because each is documented behavior that is easy to read the wrong way:

- **Redis ACL lines rebuild the user.** A configuration-file line `user default on nopass ~* &* +@read +@write` produced a user who could *not* run `CONFIG`. The analyzer treated the rules as additions to the default user's implicit `+@all`.
- **The ACL line beats `requirepass` regardless of order.** With `user default ... nopass` earlier in the file and `requirepass` later, the server still accepted unauthenticated clients. The analyzer applied the two in file order.
- **`satisfy any` needs an explicit allow.** With `auth_basic` on and access rules that did not match the client, the analyzer assumed that "no rule matched" would satisfy the access condition. Nginx answered 401.

None of these cases was planted. All arose naturally from the generator and the drift model. A twin needs no model of these rules because the server applies them itself.

F. Cost

The study ran about 42,000 distinct twin evaluations. Median wall time per evaluation, covering start, probe and stop, was 26 ms for Nginx, 79 ms for PostgreSQL, 131 ms for Redis and 161 ms for OpenSSH. The full pipeline (E0–E3) completed in about 20 minutes on four vCPUs. Process-level twins of this kind are cheap enough that ddmin cores and per-edit checks are practical inside a platform, not just in an offline study.

VI. DISCUSSION

What a twin platform should change. Four concrete changes follow for the Digital Twin platform that motivated this work. (1) *Synchronize by guard keys.* The agent can export only directives whose keys are in the union of learned and expert guard sets, about a quarter to a third of the file in our corpus. This narrows what a multi-tenant control plane holds about each customer. (2) *Ship absence.* For every guard key the agent should state "unset" explicitly, so that platform defaults never stand in for vendor defaults. Our largest residual errors came from exactly this substitution. (3) *Guard re-emulation by key, with the expert list as a floor.* Replacing per-component hashes with key guards cut re-emulation three- to five-fold at zero or near-zero staleness. Learned keys can trim the expert list per property, but they should not replace it, because a learned guard cannot know keys that never mattered in its sample. (4) *Report cores and pivots with findings.* A core is a minimal explanation ("this verdict is due to line 7"). The pivots are a remediation map ("removing line 12 would expose the service"). Both are cheap to compute on a live twin.

Why not static rules alone? For four mature services, a careful analyst can encode almost everything, and our careful analyzer did. The residual 0.08% still matters. Four of the 14 disagreements were missed exposures, where the analyzer called a Redis server safe that accepted unauthenticated administrative commands. They surfaced only after thousands of naturally varied configurations. The twin and the analyzer are complements. The analyzer is instant and explainable. The twin keeps it honest, and fidelity cores tell the twin what it may safely leave out.

Concentrated but fragile. The juxtaposition of tiny cores with a slow fidelity curve is the paper's central observation. A random 90% copy still misses the one line that matters 6.6% of the time. Tiny cores alone also make a poor synchronization guide, because necessity lives in pivots. Fidelity is a property of a verdict, not of a file.

VII. THREATS TO VALIDITY

Synthetic corpus. The configurations are generated, not collected. We modeled postures on well-known misconfiguration patterns, accepted only configurations the real servers run and released the generator. Nevertheless, real fleets may concentrate verdicts in other keys, use longer files or interact across services. Core sizes in particular depend on posture diversity. Our cores of at most two directives describe this corpus, not a law. **Narrow properties.** Each property is one wire question. Richer properties, such as reachability through a proxy chain or role-dependent SQL privileges, would likely produce larger cores and weaker locality. **Single-host emulation.** Twins are processes on one host with a shared address. Firewall, network-namespace and multi-service effects of the

earlier platform are outside this study. **Learned guards.** Guard sets were learned from 30 configurations per family and tested on 30 held-out ones from the same generator. Distribution shift would hurt them more than the expert list. That is one reason we recommend the expert list as a floor. **Analyzer authorship.** The static analyzers were written by the same team, from documentation and before seeing the comparison. A different expert would make different mistakes. We report the disagreements as examples of semantic corners, not as a ranking of tools. **Determinism.** Each distinct twin was evaluated once. To check that this is safe, we re-ran a random sample of 1,000 cached twins on fresh server processes: all 1,273 verdicts were identical. Confidence intervals therefore reflect variation across configurations, not across runs.

VIII. Conclusion

We asked how much configuration a security service twin needs, and the answer has two parts. To *reproduce* a verdict, very little: at most two directives in every case we measured. To *keep* reproducing it as configurations change, the twin must track the larger set of directives whose removal would flip it, and it must know when a key is absent. Treating fidelity as verdict-relative rather than file-relative lets a twin platform export a quarter of the configuration, re-emulate a fifth as often, and still agree with the real servers more than 99% of the time. It also explains why a twin built from defaults, or from most of the file, can be confidently wrong. The harness, corpus generator, raw results and analysis scripts are released with this paper.

References

[1] "Digital Twin: Discovery, emulation and bounded hunting," source code and evaluation artifacts, GitHub repository poojarishreyas/Digital_twin, 2026. [Online]. Available: https://github.com/poojarishreyas/Digital_twin

[2] M. Grieves and J. Vickers, "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," in *Transdisciplinary Perspectives on Complex Systems*, F.-J. Kahlen, S. Flumerfelt, and A. Alves, Eds. Cham, Switzerland: Springer, 2017, pp. 85–113.

[3] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, Apr. 2019.

[4] M. Eckhart and A. Ekelhart, "Towards security-aware virtual environments for digital twins," in *Proc. 4th ACM Workshop Cyber-Physical System Security (CPSS)*, 2018, pp. 61–72.

[5] M. Eckhart and A. Ekelhart, "Digital twins for cyber-physical systems security: State of the art and outlook," in *Security and Quality in Cyber-Physical Systems Engineering*, S. Biffl, M. Eckhart, A. Lüder, and E. Weippl, Eds. Cham, Switzerland: Springer, 2019, pp. 383–412.

[6] M. Dietz and G. Pernul, "Unleashing the digital twin's potential for ICS security," *IEEE Security & Privacy*, vol. 18, no. 4, pp. 20–27, 2020.

[7] P. Empl, D. Koch, M. Dietz, and G. Pernul, "Digital twins in security operations: State of the art and future perspectives," *ACM Comput. Surv.*, vol. 58, no. 1, Art. no. 18, 2025.

[8] H. H. Liu *et al.*, "CrystalNet: Faithfully emulating large production networks," in *Proc. 26th ACM Symp. Operating Systems Principles (SOSP)*, 2017, pp. 599–613.

[9] M. M. Yamin, B. Katt, and V. Gkioulos, "Cyber ranges and security testbeds: Scenarios, functions, tools and architecture," *Computers & Security*, vol. 88, Art. no. 101636, Jan. 2020.

[10] A. Fogel, S. Fung, L. Pedrosa, M. Walraed-Sullivan, R. Govindan, R. Mahajan, and T. Millstein, "A general approach to network configuration analysis," in *Proc. 12th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2015, pp. 469–483.

[11] R. Birkner, D. Drachsler-Cohen, L. Vanbever, and M. Vechev, "Config2Spec: Mining network specifications from network configurations," in *Proc. 17th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2020, pp. 969–984.

[12] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: A logic-based network security analyzer," in *Proc. 14th USENIX Security Symp.*, 2005, pp. 113–128.

[13] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proc. IEEE Symp. Security and Privacy*, 2002, pp. 273–284.

[14] Z. Yin, X. Ma, J. Zheng, Y. Zhou, L. N. Bairavasundaram, and S. Pasupathy, "An empirical study on configuration errors in commercial and open source systems," in *Proc. 23rd ACM Symp. Operating Systems Principles (SOSP)*, 2011, pp. 159–172.

[15] T. Xu, L. Jin, X. Fan, Y. Zhou, S. Pasupathy, and R. Talwadker, "Hey, you have given me too many knobs!: Understanding and dealing with over-designed configuration in system software," in *Proc. 10th Joint Meeting European Software Engineering Conf. and ACM SIGSOFT Symp. Foundations of Software Engineering (ESEC/FSE)*, 2015, pp. 307–319.

[16] L. Keller, P. Upadhyaya, and G. Candea, "ConfErr: A tool for assessing resilience to human configuration errors," in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2008, pp. 157–166.

[17] M. Sayagh, N. Kerzazi, B. Adams, and F. Petrillo, "Software configuration engineering in practice: Interviews, survey, and systematic literature review," *IEEE Trans. Softw. Eng.*, vol. 46, no. 6, pp. 646–673, 2020.

[18] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *Proc. 41st IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2019, pp. 164–175.

[19] C. Tang *et al.*, "Holistic configuration management at Facebook," in *Proc. 25th ACM Symp. Operating Systems Principles (SOSP)*, 2015, pp. 328–343.

[20] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Trans. Softw. Eng.*, vol. 28, no. 2, pp. 183–200, Feb. 2002.

[21] M. Weiser, "Program slicing," *IEEE Trans. Softw. Eng.*, vol. SE-10, no. 4, pp. 352–357, Jul. 1984.